
Chapter 1 | NPU 与系统架构

- 本章目标：为 NPU 扩展确定系统级设计。
- 内容：为什么需要 NPU、系统分层、MMIO 协议、int8 数据格式。
- 产出：设计文档 + 顶层端口草案。

Chapter 1 | 为什么 CPU 跑不动 CNN

计算机执行程序时，CPU 一条一条地“取指令 -> 译码 -> 执行”。Hack 的 ALU 只会做加法、按位与、取反等操作，没有乘法指令；一个乘累加 ($a*w+b$) 在 Hack 上要拆成很多条加法/移位指令，速度会非常慢。

更关键的是，CNN 的运算量是几万到几亿次 MAC。如果每个 MAC 都要经过“取指-译码-执行”，指令流本身就变成瓶颈。专用硬件可以把很多 MAC 放在同一个时钟周期里并行完成。

- Hack ALU：16 位加法与按位运算，没有乘法。
- RAM：32K x 16bit = 64KB，装不下 YOLO 权重。
- CNN 推理的本质是大量重复的乘累加 (MAC)。
- 结论：需要专用硬件做 GEMM。

Chapter 1 | Von Neumann 瓶颈

- 取指 -> 译码 -> 执行 -> 访存，每条指令都要搬数据。
- MAC 运算密度高，指令流会成为瓶颈。
- NPU 用数据流 + 局部 SRAM 降低搬运开销。

Chapter 1 | 系统分层

MMIO (Memory-Mapped I/O) 的意思是：外设的寄存器和内存使用同一条地址总线。CPU 不需要新的专用指令，只要“往某个地址写值”或“从某个地址读值”，就能控制外设。

比如地址 0x7000 可以定义为“命令寄存器”，CPU 执行 '@0x7000; M=D' 就把 D 的内容写给了 NPU。对 CPU 来说，这跟写 RAM 没有区别；对 NPU 来说，它看到地址落在自己的区间，就接管这次读写。

- Hack CPU：调度器，发起推理、轮询状态。
- Memory：保留原有 RAM/Screen/Keyboard，扩展 MMIO 段。
- NPU：命令解析、DMA、脉动阵列、激活/池化。
- 数据路径：CPU -> Memory -> NPU SRAM -> Array -> 输出。

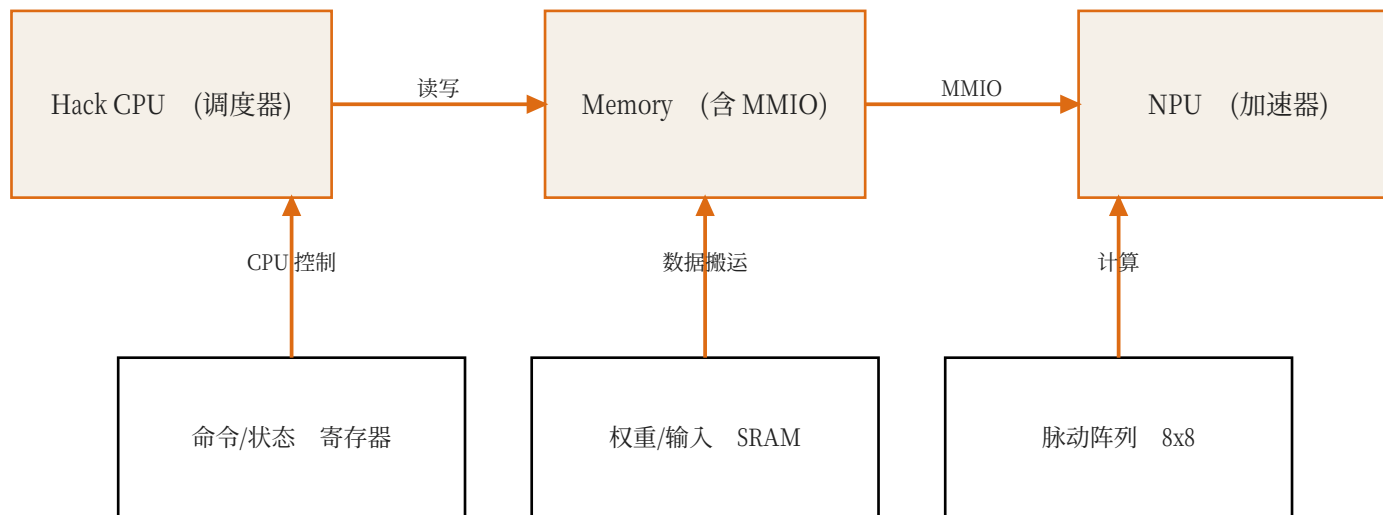


图 1 : Hack CPU 只做调度, 重计算全部下沉到 NPU

Chapter 1 | NPU 内部结构

NPU 不是一块“魔法芯片”，它内部也是寄存器、SRAM 和运算阵列的组合。命令寄存器告诉它“做什么”，状态寄存器告诉 CPU “做完没有”，权重 SRAM 存放模型参数，输入/输出 SRAM 存放特征图。

脉动阵列只负责最重的矩阵乘；周围这些“搬运工”决定了阵列能不能吃饱。这也是为什么真实 NPU 设计里 DMA 和缓存往往比阵列本身更难。

- 命令寄存器：start、layer_id、src/dst 地址。
- 状态寄存器：busy、done、error。
- 权重 SRAM：保存 int8 权重。
- 输入/输出 SRAM：保存特征图。
- 脉动阵列：8x8 起步，完成矩阵乘。

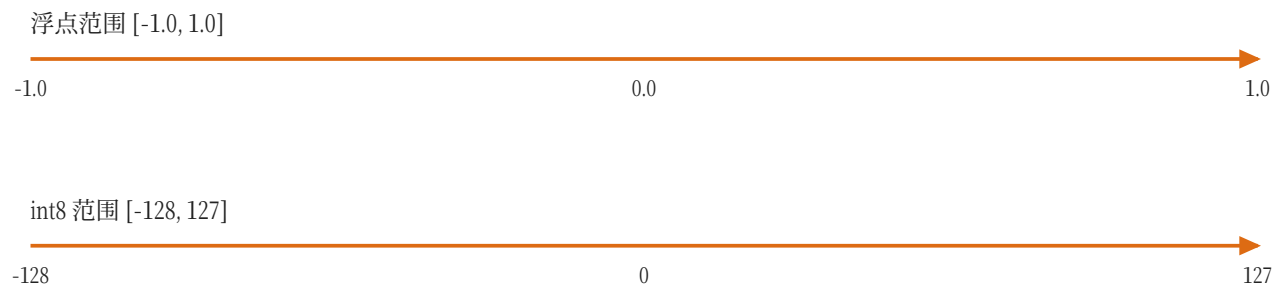
Chapter 1 | 为什么是 int8

浮点数 (float32) 做乘法需要很大的硬件，int8 乘法器只有它几分之一面积和功耗。NPU 通常把权重和激活量化成 8 位有符号整数，累加时用 32 位，保证精度。

量化不是“随便取整”：每层有一个 scale，把浮点区间映射到 $[-128, 127]$ 。这个映射关系会在 Ch4 详细讲。

- 模型权重和激活用 8 位有符号整数表示。
- int8 乘法面积小、速度快，适合 FPGA/ASIC。
- 累加器用 32 位，避免精度损失。
- 每层需要 scale 参数完成反量化。

图 4 : $q = \text{round}(\text{clamp}(x / \text{scale}))$, $\text{scale} = 1/127$ 。



Chapter 1 | GEMM 是核心算子

GEMM (General Matrix Multiply) 是 $C = A \times B$ 。卷积可以通过 im2col 变成矩阵乘，全连接本来就是矩阵乘。所以只要硬件把矩阵乘做快，CNN 的大部分计算就快起来了。

脉动阵列正是为 GEMM 设计的：它把乘法器排成网格，让数据像心跳一样在 PE 之间流动，从而在一个周期内完成很多 MAC。

- 卷积可以转成矩阵乘：im2col。
- 全连接本来就是矩阵乘。
- 脉动阵列的目标：一个周期完成 N 个 MAC。
- 后续章节围绕“如何喂数据给阵列”展开。

Paper: Kung, Why Systolic Architectures?, IEEE Computer, 1982

Chapter 1 | MMIO 协议

一次最简单的 MMIO 交互分三步：第一，CPU 把输入数据地址、输出地址、层参数写进 NPU 的寄存器；第二，CPU 向 CMD 寄存器写 START；第三，CPU 反复读 STATUS 寄存器，直到 DONE 位置 1，再读取结果。

寄存器表就是 CPU 与 NPU

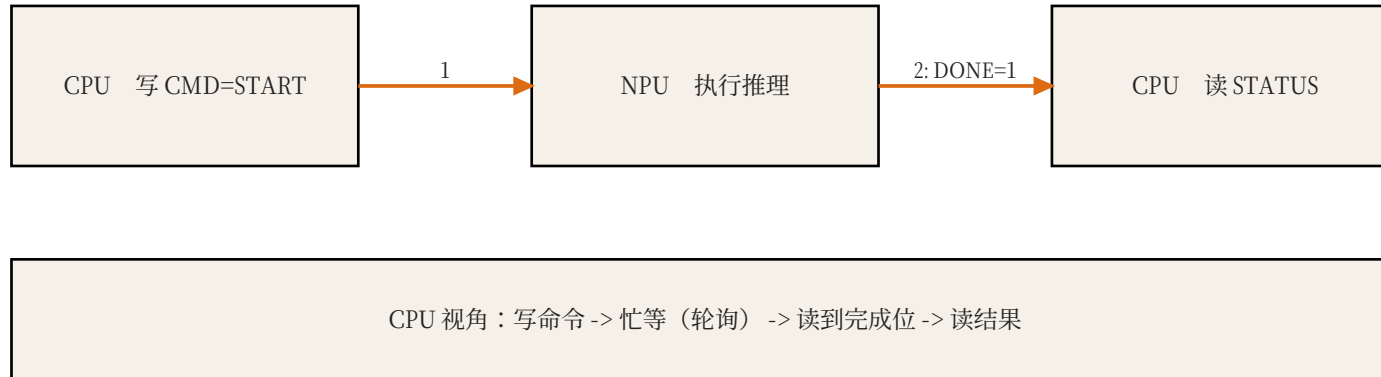
之间的“合同”：地址、位宽、读写方向、含义都必须提前定好。寄存器表不清楚，后面写驱动时就会互相猜。

- 地址 0x7000-0x7FFF 预留给 NPU。
- CPU 写 CMD 寄存器启动推理。
- CPU 读 STATUS 寄存器轮询完成。
- 数据缓冲区放在共享 RAM 或 NPU 私有 SRAM。

图 2：Hack 地址空间。0x7000 以上原本未使用，
正好预留给 NPU。



图3：MMIO 的本质就是“把外设寄存器当成内存地址来读写”。



Paper: Patterson & Hennessy, Computer Organization and Design, memory-mapped I/O 章节

Chapter 1 | 数据流总览

端到端的数据流是：图片从 RAM 拷到输入 SRAM，权重从权重 SRAM 装载进阵列；控制器逐像素把数据送进阵列，阵列输出经过激活/池化后写回输出 SRAM；最后一层结束后，CPU 把结果读回 RAM。

这条路径上的每一步都是后面章节的作业：Ch2 做阵列，Ch3 做控制器，Ch4 做权重，Ch5 做整机。

- 图片 -> 输入 SRAM（或共享 RAM）。
- 权重 -> 权重 SRAM -> 脉动阵列。
- 阵列输出 -> 激活/池化 -> 输出 SRAM。
- 最终结果写回 Hack RAM，CPU 读取。

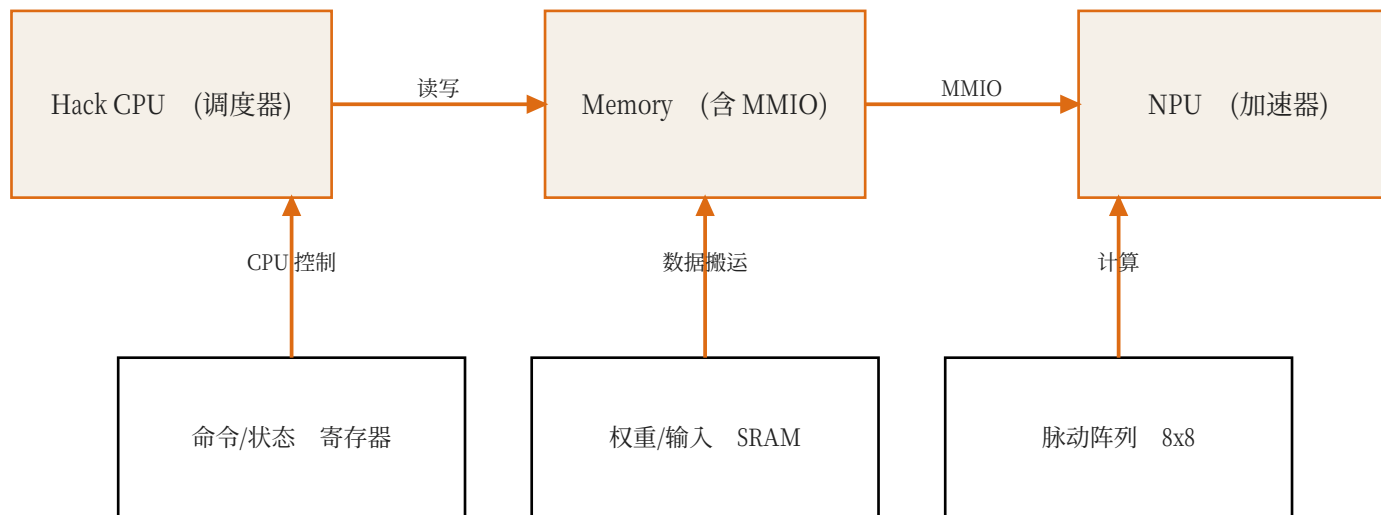


图 1 : Hack CPU 只做调度, 重计算全部下沉到 NPU

Chapter 1 | 本章任务

- 写系统设计文档。
- 画系统框图并标注数据流。
- 设计寄存器表与命令描述符。
- 给出 8x8 阵列的吞吐量估算。

Chapter 1 | 路线图

- Ch2：PE 与 SystolicArray（已搭好框架）。
- Ch3：GEMM 控制器与卷积数据通路。
- Ch4：模型训练、量化与导出。
- Ch5：Hack 整机集成与端到端推理。

Chapter 1 | 总结

- NPU 的价值：把重复 MAC 从 CPU 卸载到专用硬件。
- MMIO 是 CPU 与 NPU 之间的“合同”。
- `int8 + int32` 累加是本课程统一的数值格式。
- 下一章从单个 PE 开始搭建阵列。

术语速查 (Glossary)

- CPU：中央处理器，负责执行指令；在本课程里是 Hack CPU。
- RAM：随机存取存储器，Hack 有 64KB，用于放数据和程序状态。
- MMIO：Memory-Mapped I/O，把外设寄存器映射到内存地址，CPU 用普通读写指令控制外设。
- 寄存器：CPU 或外设内部的小容量存储单元，通常 8/16/32 位。
- SRAM：片上静态随机存储器，速度快、容量小，NPU 用它暂存权重和特征图。
- MAC：乘累加运算： $acc = acc + a * w$ ，是 CNN 最基础的计算。
- GEMM：通用矩阵乘， $C = A \times B$ ；卷积可以转换成 GEMM。
- PE：Processing Element，处理单元；本课程里是一个 MAC 单元。
- Systolic Array：脉动阵列：PE 排成网格，数据在相邻 PE 间逐拍流动。
- Weight-Stationary：权重驻留式数据流：权重装载后停在 PE 内反复使用。
- 斜输入：把第 row 行输入延迟 row 拍，使阵列各行在同一拍处理同一个 k。
- im2col：Image to Column，把卷积窗口展平成矩阵行，从而用 GEMM 计算卷积。
- Line Buffer：行缓冲：只保留最近若干行，供滑动窗口复用。
- FSM：有限状态机：一组状态和跳转条件，NPU 控制器用它实现调度。
- DMA：Direct Memory Access，直接内存访问，批量搬运数据。

-
- 量化：把浮点数映射到低比特整数（如 int8），并记录 scale 以便还原。
 - Scale：量化比例因子，决定一个整数单位代表多大的浮点数值。
 - Polling：轮询：CPU 反复读状态寄存器，直到外设完成。
 - argmax：取最大值的下标；分类任务里就是预测类别。
 - NPU：Neural Processing Unit，神经网络处理器，专为张量运算设计的加速器。

References / 延伸阅读

- Kung, H.T., "Why Systolic Architectures?", IEEE Computer, 1982.
- Jouppi et al., "In-Datacenter Performance Analysis of a Tensor Processing Unit", ISCA 2017.
- Sze et al., "Efficient Processing of Deep Neural Networks: A Tutorial and Survey", Proceedings of the IEEE, 2017.
- Patterson & Hennessy, "Computer Organization and Design" (MMIO / I/O 章节) .